

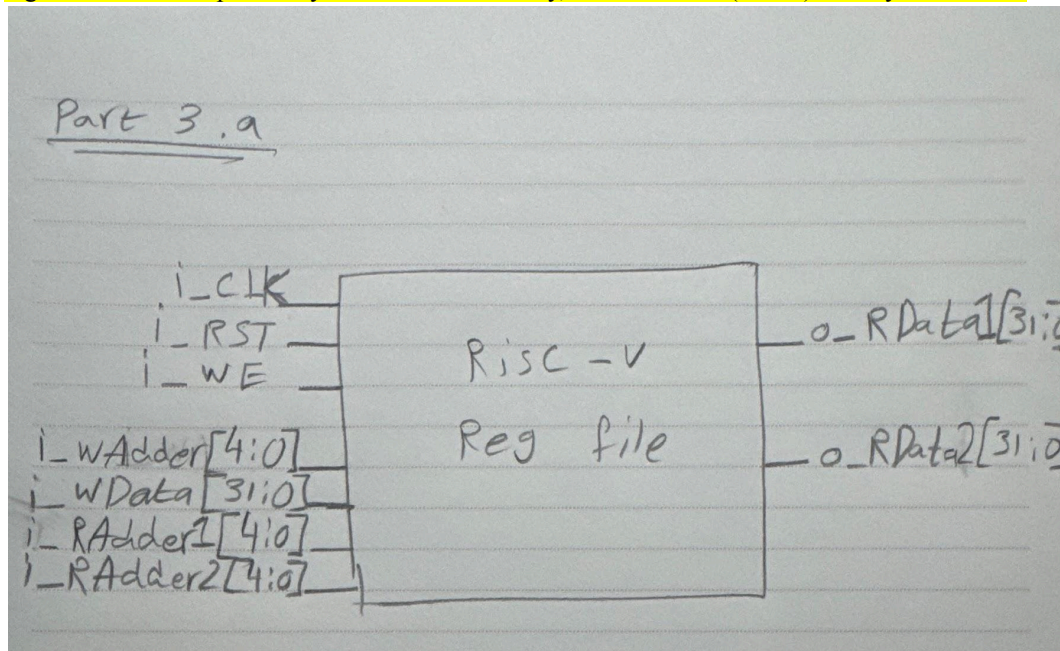
CprE 381, Computer Organization and Assembly-Level Programming

Lab 2 Report

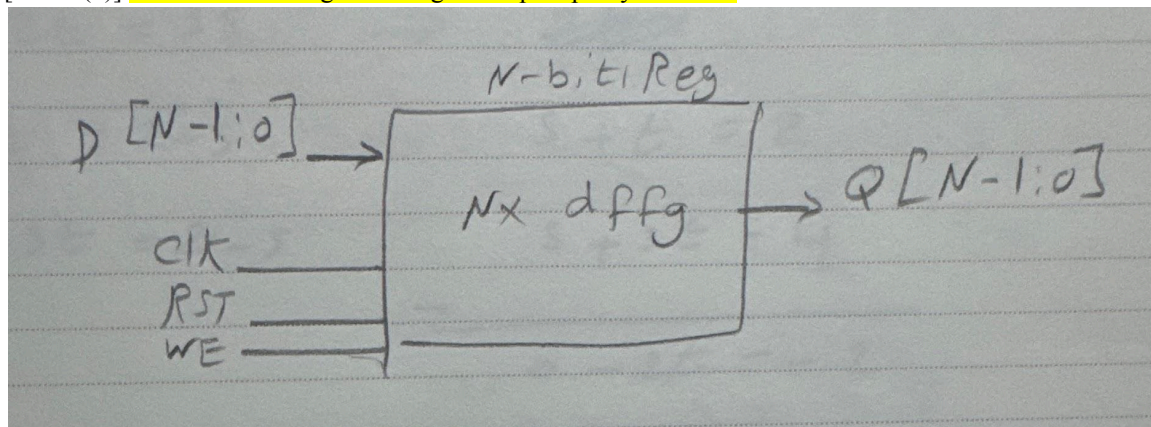
Student Name _____ Tariq Almaaini _____

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

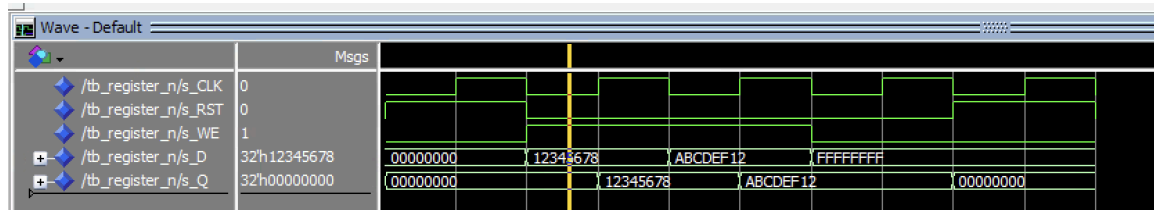
[Part 3 (a)] Draw the interface description (i.e., the “symbol” or high-level blackbox) for the RISC-V register file. Which ports do you think are necessary, and how wide (in bits) do they need to be?



[Part 3 (b)] Create an N-bit register using this flip-flop as your basis.



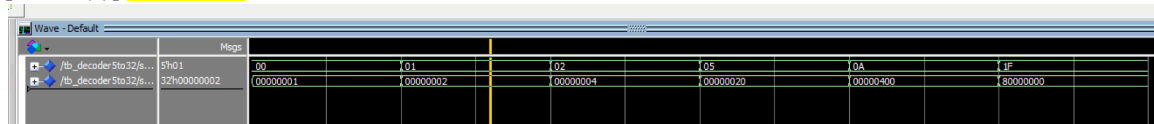
[Part 3 (c)] **Waveform.**



[Part 3 (d)] **What type of decoder would be required by the RISC-V register file and why?**

- A 5-to-32 decoder is required because the register file has 32 registers. The 5-bit write address is decoded to select which one of the 32 registers will be written to.

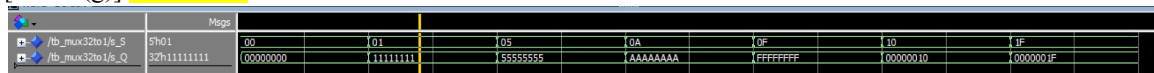
[Part 3 (e)] **Waveform.**



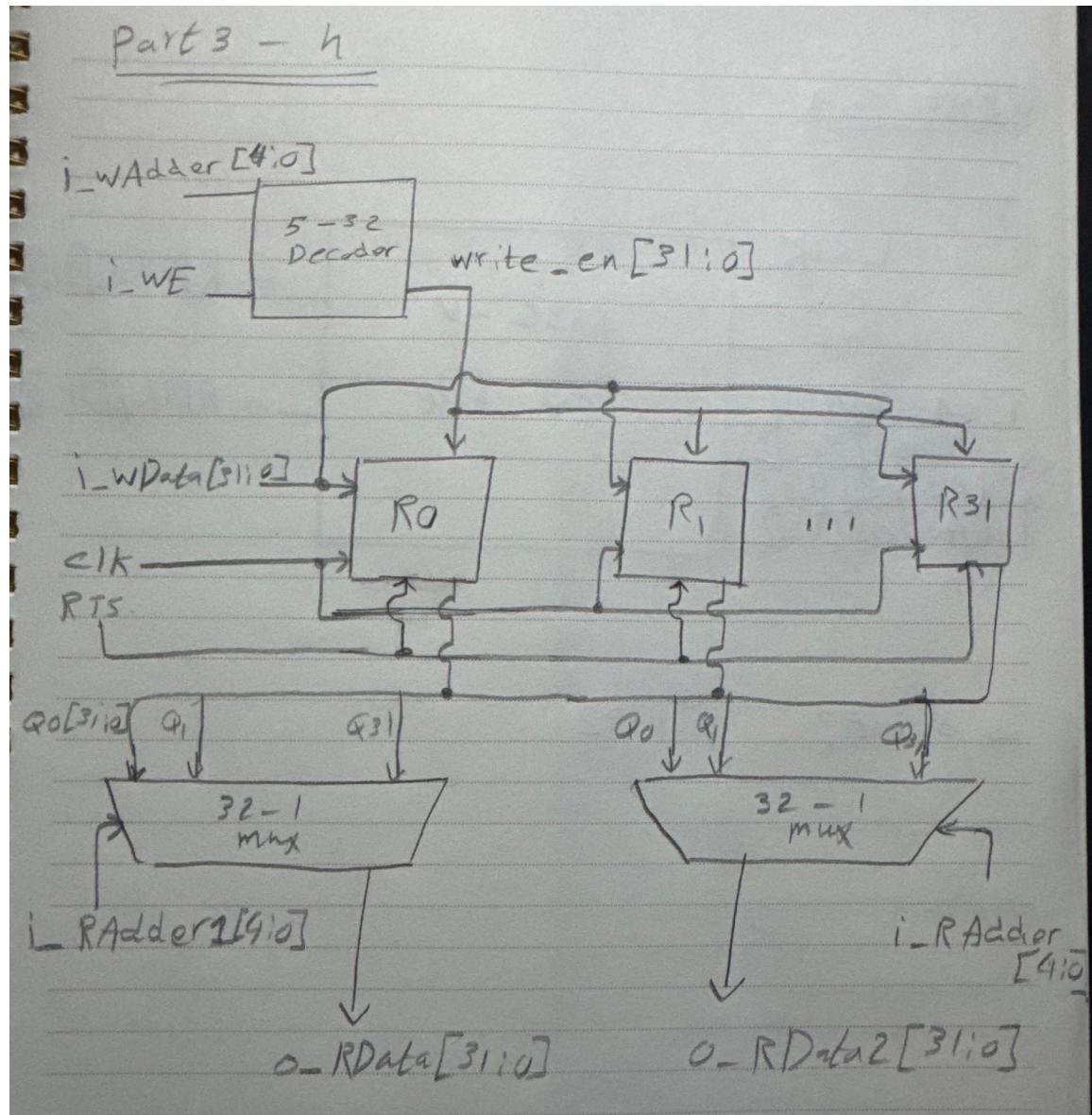
[Part 3 (f)] **In your write-up, describe and defend the design you intend on implementing for the next part.**

- The register file will use 32 32-bit registers, a 5-to-32 decoder to select the register being written to, and two 32-to-1 multiplexers to select the two registers being read. This design allows one register to be written while two registers can be read independently.

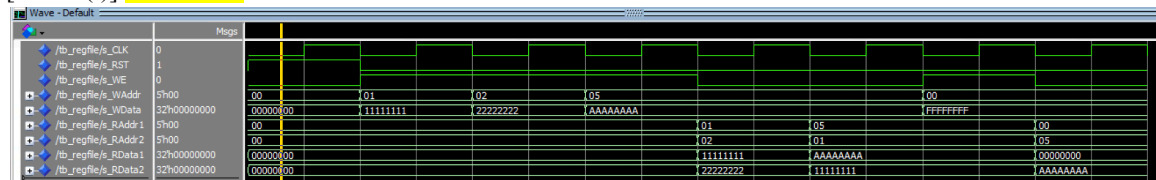
[Part 3 (g)] **Waveform.**



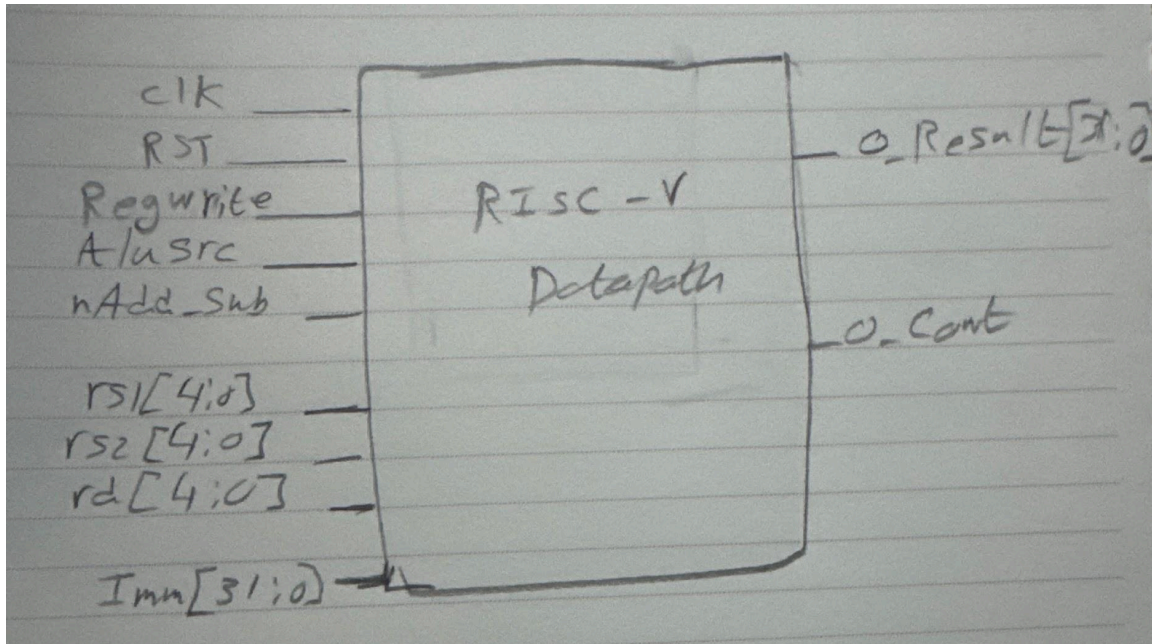
[Part 3 (h)] **Draw a (simplified) schematic (i.e., components within the high-level blackbox) for the RISC-V register file, using the same top-level interface ports as in your solution describe above and using only the register, decoder, and mux VHDL components you have created.**



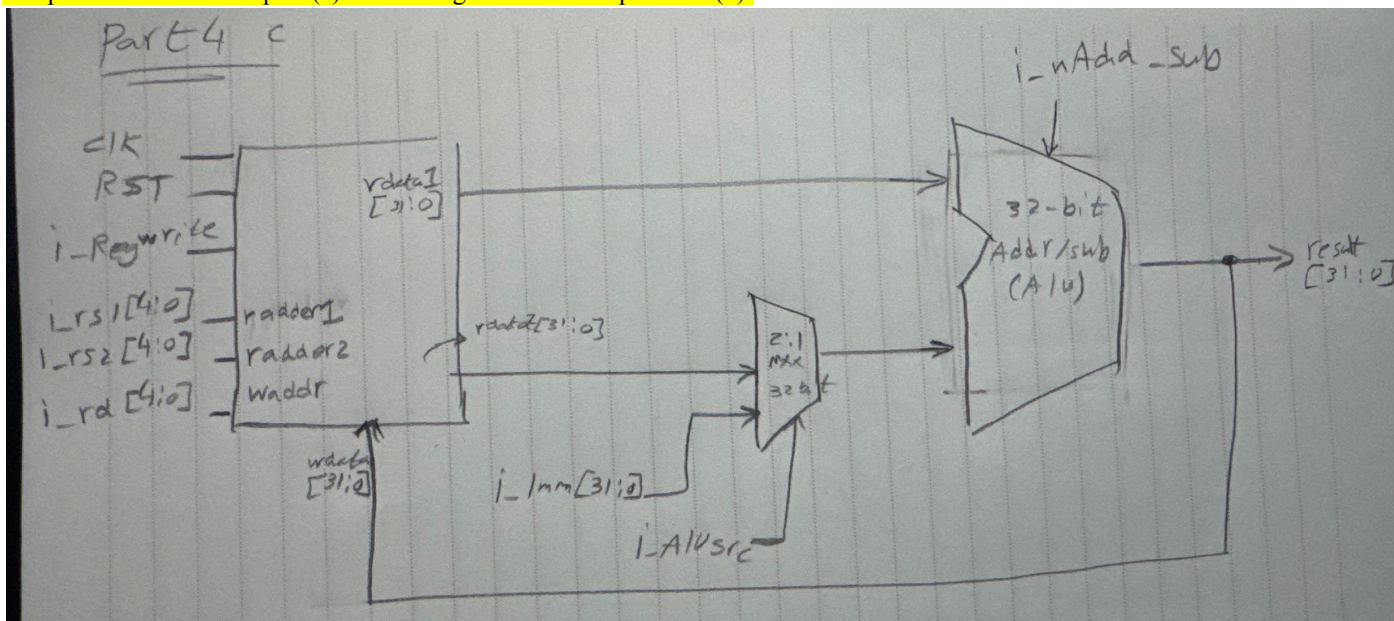
[Part 3 (i)] **Waveform.**



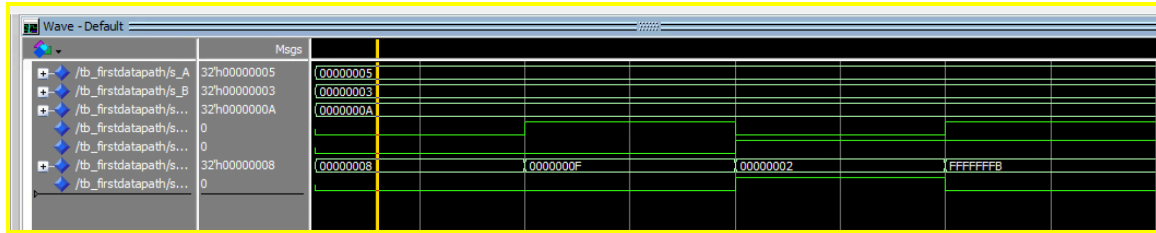
[Part 4 (b)] **Draw a symbol for this RISC-V-like datapath.**



[Part 4 (c)] Draw a schematic of the simplified RISC-V processor datapath consisting only of the component described in part (a) and the register file from problem (1).



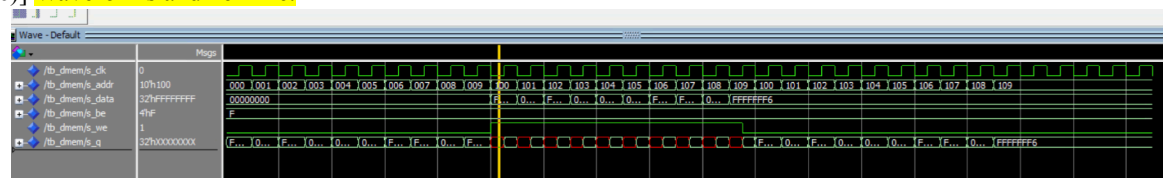
[Part 4 (d)] Include in your report waveform screenshots that demonstrate your properly functioning design. Annotate what the final state of the register file should be and provide a mapping of the registers used in your drawing to the register names. Include a screenshot of RARs output and compare it against your output.



[Part 5 (a)] Read through the mem.vhd file, and based on your understanding of the VHDL implementation, provide a 2-3 sentence description of each of the individual ports (both generic and regular).

- The mem.vhd file uses DATA_WIDTH, ADDR_WIDTH, and BYTE_WIDTH to define the memory word size, address size, and byte size. The clk, addr, data, be, and we ports control memory access and writing, while q outputs the data stored at the selected address. Writes occur using the clock and byte-enable signals, while the selected memory value is provided through q.

[Part 5 (c)] Waveforms and hex file.

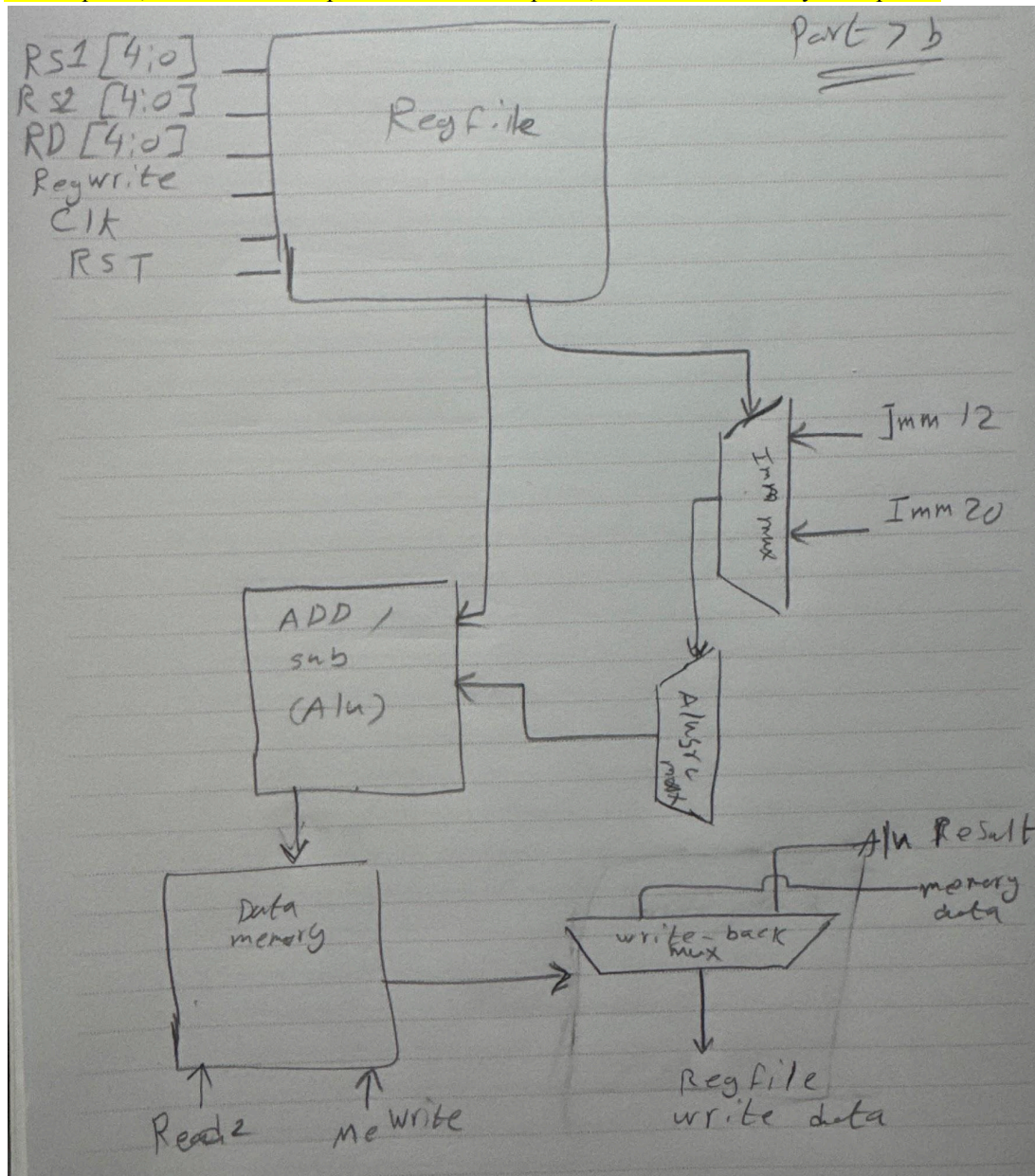


Wave - Default		Mem		Data		Addr		Data		Addr		Data	
tb_extenders/Imm	12h001	001		fff		fff		fff		fff		fff	
tb_extenders/Imm	12h0000001	00000001		00000fff		00000fff		00000fff		00000fff		00000fff	
tb_extenders/Imm	12h00000001	00000001		00000fff		00000fff		00000fff		00000fff		00000fff	

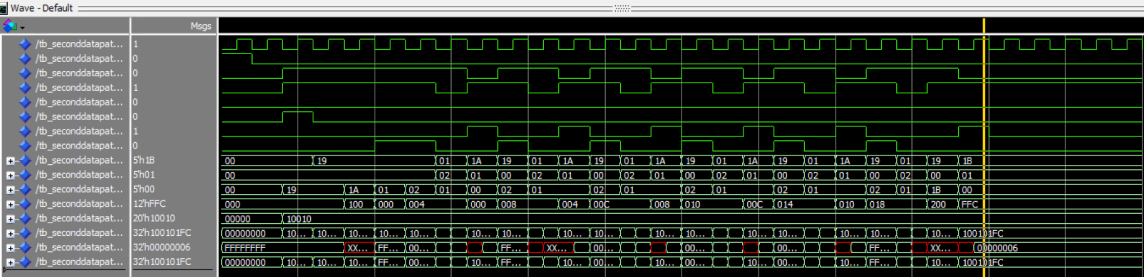
[Part 7 (a)] what control signals will need to be added to the simple processor from part 2? How do these control signals correspond to the ports on the mem.vhd component analyzed in part 3?

- MemWrite and MemtoReg are added to the processor. MemWrite controls the we port of mem.vhd to enable memory writes, while MemtoReg selects whether the value written back to the register file comes from memory or the ALU. The be port is set to "1111" for word operations so all four bytes are enabled.

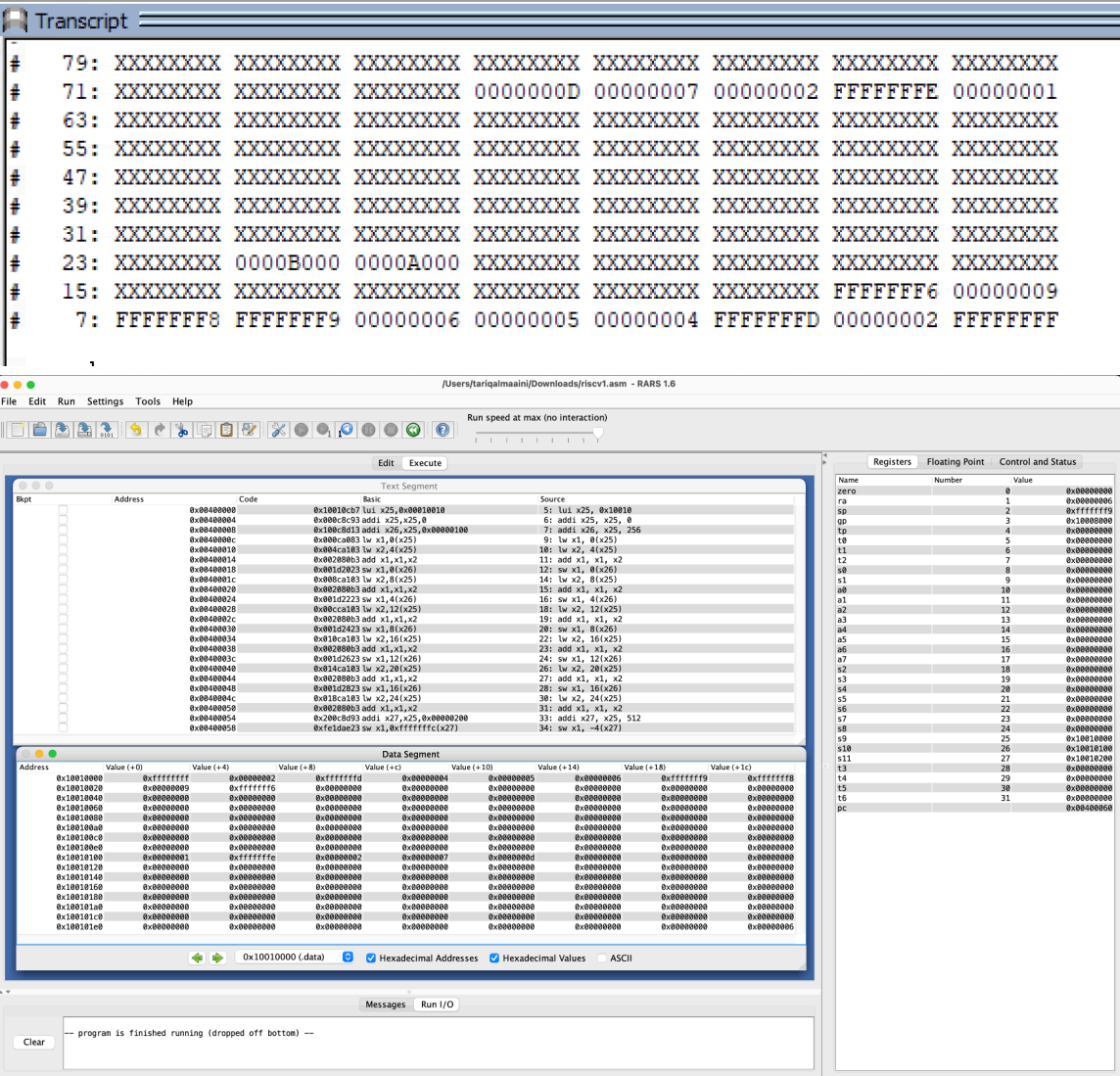
[Part 7 (b)] Draw a schematic of a simplified RISC-V processor consisting only of the base components used in part 2, the extender component described in part 4, and the data memory from part 3.



Waveform.



Compare the Questasim waveform/memory list against the RARs output/memory list.



- QuestaSim and RARS produced the same memory results. The output array B contains 1, -2, 2, 7, and 13, and B[63] contains 6 (0x00000006). This confirms that the datapath produced the expected results.